🤖 V5 Pre-Build Agent Toolkit

Build these agents **before** writing V5 feature code. They form the guardrails, scaffolding, and CI pipeline that every AI agent (Ralph Loop) depends on.

> **Rule:** P0 must be green before any Layer 0 code is written. P1 unlocks the Ralph Loop. P2 completes during active development.

P0 — Blocking (build in 2 days, ~20 hrs)

> Nothing starts without these. They encode architecture rules so Claude Code can't violate them.

1. CLAUDE.md + Layer Convention Encoder

- [ ] Create root `CLAUDE.md` under 150 lines using WHAT/WHY/HOW structure
- [ ] Add path-scoped rules: `src/kernel/**` → Layer 0 only, `src/runtime/**` → Layer 3 only, etc.
- [ ] Create `.claude/rules/` folder with one `.md` per layer (L0–L6 + 4A sub-layers)
- [ ] Reference the 7-layer build order from Master Build Plan
- [ ] Add import boundary rules: each layer may only import from layers below it
- [ ] Add naming conventions from Terminology doc

Effort: 3–4 hrs | **Deps:** None | **Tooling:** Claude Code skills system

---

2. Zod Schema Registry

- [ ] Install `zod` v4+
- [ ] Create `src/kernel/schemas/` directory
- [ ] Define base `WidgetManifest` schema (matches Interface Contracts doc)
- [ ] Define `BusEvent` base schema (type, payload, optional `spatial?: SpatialContext`)

- [ ] Define `DataSource` schema (id, type, ownerId, scope, canvasId?)
- [ ] Define `CanvasEntity` base schema
- [ ] Export JSON Schema via `z.toJSONSchema()` for plugin manifests
- [ ] All schemas re-export from `src/kernel/schemas/index.ts`

Effort: 3–4 hrs | **Deps:** Agent #1 | **Tooling:** `zod` v4, `zod-to-json-schema`

---

3. ESLint Layer Boundary Enforcer

- [ ] Install `eslint-plugin-boundaries`
- [ ] Define element types matching 7-layer directory structure in `.eslintrc`
- [ ] Set `default: "disallow"` — all cross-layer imports fail unless explicitly allowed
- [ ] Add allow rules for each valid layer dependency direction
- [ ] Install `eslint-plugin-import` with `no-cycle` rule
- [ ] Verify: a fake import from Layer 3 into Layer 0 throws ESLint error
- [ ] Add ESLint MCP server: `npx @eslint/mcp@latest` to Claude Code config

Effort: 2–3 hrs | **Deps:** Agent #1 | **Tooling:** `eslint-plugin-boundaries`, `@eslint/mcp`

---

4. dependency-cruiser Validator

- [ ] Install `dependency-cruiser`
- [ ] Configure `.dependency-cruiser.js` with `forbidden` rules for each layer pair
- [ ] Add 7 rules enforcing downward-only dependencies
- [ ] Add 1 global rule forbidding circular imports
- [ ] Add `depcruise --output-type err` as a CI script
- [ ] Add SVG diagram generation: `depcruise --output-type dot | dot -T svg > arch.svg`
- [ ] Commit arch.svg to `docs/` for visual reference

Effort: 2–3 hrs | **Deps:** Agents #1, #3 | **Tooling:** `dependency-cruiser` v17+, GraphViz

---

5. Vitest Test Harness

- [ ] Install `vitest`, `@vitest/coverage-v8`, `@vitest/browser`
- [ ] Install `vitest-canvas-mock` for HTMLCanvasElement mocking (Layer 4A tests)
- [ ] Configure `vitest.config.ts`: dual environments (jsdom for L0-3/5-7, browser for L4)
- [ ] Set coverage thresholds: 80% branches, functions, lines, statements across all layers
- [ ] Configure `bench()` API for event bus throughput benchmarks (L0 kernel)
- [ ] Create Claude Code skill: `.claude/commands/generate-tests.md` — reads source, generates layer-appropriate test boilerplate
- [ ] Add `npm test` and `npm run test:coverage` scripts

Effort: 2–3 hrs | **Deps:** Agent #2 | **Tooling:** `vitest`, `vitest-canvas-mock`, `@vitest/coverage-v8`

---

6. Git Hooks Pipeline

- [ ] Install `husky` v9+, `lint-staged` v15+
- [ ] Configure lint-staged: ESLint fix on `.ts/.tsx`, Prettier on all files, depcruise on `src/**/*.ts`
- [ ] Set `--max-warnings=0` on ESLint (zero tolerance)
- [ ] Add `husky install` to `prepare` npm script
- [ ] Verify: a layer violation at commit is blocked with a clear error message

Effort: 30–45 min | **Deps:** Agents #3, #4 | **Tooling:** `husky` v9, `lint-staged` v15, `prettier` v3

---

7. Core CI/CD Pipeline (GitHub Actions)

- [ ] Create `.github/workflows/ci.yml`
- [ ] Jobs: typecheck → lint → test (Node 20/22 matrix) → build → conditional deploy

- [ ] Add caching for `node_modules` and ESLint cache
- [ ] Add concurrency group to cancel stale runs on same branch
- [ ] Add `dorny/paths-filter` for selective Supabase migration runs
- [ ] No PR merges with failing CI (branch protection rule)
- [ ] Connect Vercel and Supabase to GitHub (via their GitHub integrations)

Effort: 3–4 hrs | **Deps:** Agents #3, #4, #5 | **Tooling:** GitHub Actions, `actions/setup-node@v4`

---

8. Widget Scaffolding Skill

- [ ] Create `.claude/skills/scaffold-widget/SKILL.md` with YAML frontmatter
- [ ] Skill generates: TypeScript widget class implementing `IWidget`, Zod schema, event bus registration, barrel export update, and test file
- [ ] Skill places files in correct layer directory based on widget type
- [ ] Create matching Plop.js generator in `plopfile.js` with Handlebars templates
- [ ] Test: run `/scaffold-widget Counter` → all 5 files generated correctly in right paths

Effort: 3–4 hrs | **Deps:** Agents #1, #2 | **Tooling:** `plop` v4, Claude Code skills, Handlebars

---

P1 — First Sprint (unlock Ralph Loop, ~30 hrs)

> Build alongside early Layer 0 development. Agent #9 (Ralph Loop) is the priority — it changes how fast everything else gets built.

9. Ralph Loop Harness

- [ ] Create `.ralph/` directory with `stories.json`, `progress.txt`, `learnings.txt`
- [ ] Write `.ralph/stories.json` template: array of user stories with pass/fail status
- [ ] Write bash loop script using `claude --dangerously-skip-permissions --print`

- [ ] Each iteration: reads next failing story → implements → runs `npm test` → commits if passing → updates progress
- [ ] Add Docker isolation option for unsafe operations
- [ ] Seed `stories.json` with Layer 0 kernel stories from Master Build Plan
- [ ] Verify loop runs 3 iterations autonomously

Effort: 3–4 hrs | **Deps:** Agents #5, #6, #7 | **Tooling:** `open-ralph-wiggum` CLI, bash, Docker

---

10. Plop.js Full Generator Suite

- [ ] Extend `plopfile.js` with generators for every StickerNest artifact type:
  - [ ] Event bus channel (Layer 0)
  - [ ] Zustand store (Layer 0)
  - [ ] Supabase migration file
  - [ ] Widget bridge message handler (Layer 3)
  - [ ] Canvas entity type (Layer 4A)
  - [ ] Canvas renderer (Layer 4A — 2D, 3D, VR variants)
  - [ ] Pipeline node (Layer 4A-3)
  - [ ] Plugin manifest (Layer 5/6)
- [ ] Each generator enforces correct file placement, naming, and import patterns
- [ ] Add Inquirer prompts for: target layer, rendering mode (2D/3D/VR), capabilities
- [ ] Add barrel export injection via Plop `append` action

Effort: 2–3 hrs | **Deps:** Agent #8 | **Tooling:** `plop` v4

---

11. Architecture MCP Query Server

- [ ] Extend existing StickerNest Dev MCP (currently 17 tools) with 5 new tools:
  - [ ] `architecture-check` — runs dependency-cruiser, returns violations as JSON

- ☐ `layer-map` — uses ts-morph to map every file to its architecture layer
- ☐ `dependency-graph` — runs madge, returns module tree as JSON
- ☐ `schema-registry` — lists all Zod widget schemas with inferred types
- ☐ `codebase-stats` — file counts, export counts, test coverage per layer

☐ Wire tool backends: dependency-cruiser, madge, ts-morph

☐ Expose as MCP server consumed by Claude Code

☐ Test: ask Claude Code "does `src/runtime/bridge.ts` violate layer rules?" → gets JSON answer

Effort: 8–12 hrs | **Deps:** Agents #4, #2 | **Tooling:** `@modelcontextprotocol/sdk`, `ts-morph` v27, `madge` v8

---

12. Playwright Canvas E2E Agent

☐ Install `@playwright/test`

☐ Configure `playwright.config.ts` with `--use-gl=swiftshader` args (deterministic GPU-free rendering in CI)

☐ Create test helpers: canvas readiness detection via `data-ready` attribute, sticker interaction simulation

☐ Configure `maxDiffPixelRatio: 0.03` for anti-aliasing tolerance in visual tests

☐ Use Playwright Clock API to freeze animation frames for deterministic snapshots

☐ First test suite: canvas load → place sticker → verify render → widget launch flow

☐ Add `@playwright/mcp` MCP server to Claude Code config for AI-assisted test generation

Effort: 3–4 hrs | **Deps:** Agents #5, #7 | **Tooling:** `@playwright/test`, `@playwright/mcp`

---

13. Event Bus Integration Test Generator

- [ ] Build `EventRecorder` utility: subscribes to all bus channels, captures traffic for assertion
- [ ] Build test harness factory: instantiates two widgets with a shared bus
- [ ] Create Claude Code skill: `.claude/commands/generate-bus-test.md` — reads two widget types, generates full integration test
- [ ] Test pattern: Widget A emits → EventRecorder captures → assert Widget B updated state
- [ ] Include `vi.useFakeTimers()` for debounced event testing
- [ ] Include spatial event tests: VR-emitted event with `spatial` field → 2D widget handles it correctly

Effort: 2–3 hrs | **Deps:** Agents #5, #2 | **Tooling:** `vitest` spy APIs, `vi.useFakeTimers()`

---

14. Storybook Widget Stories

- [ ] Run `npx storybook@latest init` with Vite/React config
- [ ] Add `@storybook/experimental-addon-test` (Vitest addon — stories run as tests in CI)
- [ ] Add `msw-storybook-addon` for mocking Supabase API calls in stories
- [ ] Configure `tags: ['autodocs']` globally — all widget stories get auto-generated docs
- [ ] Create story templates for each widget type (emits events, receives config, themed)
- [ ] Add `npx vitest --project=storybook` to CI pipeline
- [ ] Deploy Storybook static build to Vercel as widget catalog site

Effort: 2–3 hrs | **Deps:** Agents #8, #5 | **Tooling:** `storybook` v8.5+, `msw-storybook-addon`

---

15. Supabase Migration Runner

- [ ] Add `supabase/setup-cli@v1` action to CI workflow
- [ ] Add `dorny/paths-filter@v2` — only run migrations when `supabase/migrations/**` changed

☐ Enable Supabase GitHub integration for automatic preview branch DB creation on PRs

☐ Add post-migration step: `supabase gen types typescript > src/kernel/types/supabase.ts`

☐ Verify: open a PR with a migration → Supabase branch DB auto-created, types regenerated

Effort: 1–2 hrs | **Deps:** Agent #7 | **Tooling:** Supabase CLI, `supabase/setup-cli@v1`

---

16. Vercel Preview Deployer

☐ Enable Vercel Git integration (auto-preview on every PR)

☐ Add GitHub Actions step: fetch Supabase branch credentials → inject as Vercel preview env vars

- `NEXT_PUBLIC_SUPABASE_URL` → branch URL
- `NEXT_PUBLIC_SUPABASE_ANON_KEY` → branch anon key

☐ Add PR comment posting: preview URL + Supabase branch DB URL

☐ Production deploys: triggered only on `main` merge after staging sign-off (matches Master Build Plan branch strategy)

☐ Verify: open a PR → get a preview URL that connects to an isolated Supabase branch DB

Effort: 1–2 hrs | **Deps:** Agents #7, #15 | **Tooling:** Vercel Git Integration, `@vercel/sdk`

---

17. commitlint Enforcer

☐ Install `@commitlint/cli` + `@commitlint/config-conventional`

☐ Add `.husky/commit-msg` hook

☐ Add CI step that lints all commits on a PR

☐ Scope conventions for StickerNest layers: `feat(kernel):`, `fix(runtime):`, `feat(canvas-core):`, etc.

☐ Add to CLAUDE.md so Ralph Loop commits use correct format

Effort: 30 min | **Deps:** Agent #6 | **Tooling:** `@commitlint/cli`, `@commitlint/config-conventional`

P2 — First Month (mature the environment, ~35 hrs)

> Build these while actively developing V5 layers. They add documentation, visual regression, mutation testing, and release automation.

18. Visual Regression Testing

- [ ] Add Playwright `toHaveScreenshot()` tests for canvas rendering (2D and 3D)
- [ ] Create fixed test scenes: known widget layout, fixed camera position for Three.js, frozen animations
- [ ] Baselines committed to git (`tests/snapshots/`)
- [ ] CI: fail if screenshot diff exceeds `maxDiffPixelRatio: 0.03`
- [ ] Cover: sticker render, widget container frame, pipeline wiring graph, theme token rendering

Effort: 3–4 hrs | **Deps:** Agents #12, #14 | **Tooling:** Playwright built-in snapshots

---

19. Coverage Gap Detector (Mutation Testing)

- [ ] Configure Vitest coverage with `coverage.all: true` (includes files with no tests)
- [ ] Install `@stryker-mutator/core`, `@stryker-mutator/vitest-runner`, `@stryker-mutator/typescript-checker`
- [ ] Configure Stryker scoped to critical paths: event bus, bridge protocol, Zod schemas, store logic
- [ ] Create Claude Code skill: `.claude/commands/kill-mutants.md` — reads Stryker report → generates tests for surviving mutants
- [ ] Run Stryker weekly in CI (not every commit — it's slow)

Effort: 2–3 hrs | **Deps:** Agent #5 | **Tooling:** `@stryker-mutator/core`, `@stryker-mutator/vitest-runner`

20. TypeDoc API Documentation

- [ ] Install `typedoc` + `typedoc-plugin-markdown`
- [ ] Configure entry points per layer in `typedoc.json`
- [ ] Add `eslint-plugin-tsdoc` to enforce TSDoc comment coverage on all public exports
- [ ] Create Claude Code skill: `.claude/commands/add-tsdoc.md` — batch-adds TSDoc to undocumented exports
- [ ] Add `npm run docs` to CI
- [ ] Deploy HTML output to Vercel as static docs site (`docs.stickernest.app`)

Effort: 2 hrs | **Deps:** None | **Tooling:** `typedoc` v0.28+, `typedoc-plugin-markdown`, `eslint-plugin-tsdoc`

---

21. Mermaid Architecture Diagrammer

- [ ] Use `dependency-cruiser --output-type mermaid` for live dependency graphs
- [ ] Build ts-morph script that generates Mermaid flowcharts of layer import relationships
- [ ] Create Mermaid templates for: event bus sequence diagrams, widget lifecycle flowcharts, Supabase ER diagram, Canvas entity hierarchy
- [ ] Add Mermaid MCP server (`@narasimhaponnada/mermaid-mcp-server`) to Claude Code config
- [ ] Diagrams render natively in GitHub README and Notion — commit to `docs/diagrams/`

Effort: 3–4 hrs | **Deps:** Agent #11 | **Tooling:** Mermaid.js, dependency-cruiser Mermaid output, ts-morph

---

22. Notion Doc Sync Pipeline

- [ ] Write Node.js sync script using `@notionhq/client` SDK

- [ ] Sync TypeDoc JSON output → Notion API Reference pages (under StickerNest project)
- [ ] Sync `docs/adr/` markdown → Notion ADR database
- [ ] Sync `docs/diagrams/*.md` Mermaid → Notion architecture pages (as code blocks)
- [ ] Run sync in CI after doc generation steps
- [ ] Target: existing StickerNest Notion pages updated automatically on every merge to `main`

Effort: 3–4 hrs | **Deps:** Agents #20, #21 | **Tooling:** `@notionhq/client`, Notion MCP

---

23. ADR Generator

- [ ] Install `log4brains` for ADR management
- [ ] Create `docs/adr/` directory with MADR template
- [ ] Create Claude Code skill: `.claude/commands/create-adr.md` — prompts context/options/decision → generates MADR markdown
- [ ] Seed with ADRs for decisions already made in Master Build Plan (conflict resolution strategy, widget hot-reload policy, etc.)
- [ ] Deploy `log4brains` static site to Vercel (`adr.stickernest.app`)

Effort: 2–3 hrs | **Deps:** Agent #1 | **Tooling:** `log4brains` v1.1, MADR template

---

24. Release Automation

- [ ] Add `googleapis/release-please-action@v4` to GitHub Actions (triggered on `main` push)
- [ ] Configure `release-type: node`
- [ ] Add conditional deploy job triggered by `release_created` output → fires production Vercel deploy and Supabase migration push
- [ ] Release-please maintains CHANGELOG.md automatically from conventional commits

- [ ] Verify: merge a `feat(kernel):` commit → Release PR auto-created with version bump

Effort: 1–2 hrs | **Deps:** Agents #17, #7 | **Tooling:** `release-please-action@v4`

---

25. ts-morph Deep Codebase Analyzer

- [ ] Build analysis scripts using `ts-morph` v27:
  - [ ] Validate all widget classes implement `IWidget` lifecycle interface
  - [ ] Check Zod schemas match their corresponding TypeScript types
  - [ ] Extract all event bus channel registrations across codebase
  - [ ] Map every export to its architecture layer
- [ ] Expose analysis as new tools in Architecture MCP Query Server (Agent #11)
- [ ] Use to detect: widgets missing `manifest`, events emitted without schema, stores importing across layers

Effort: 4–6 hrs | **Deps:** Agent #11 | **Tooling:** `ts-morph` v27

Build Order Summary

#	Agent	Hrs	Status
1	CLAUDE.md • Layer Rules	3–4	☐
2	Zod Schema Registry	3–4	☐
3	ESLint Boundary Enforcer	2–3	☐
4	dependency-cruiser	2–3	☐
5	Vitest Test Harness	2–3	☐
6	Git Hooks Pipeline	0.5	☐
7	Core CI/CD Pipeline	3–4	☐
8	Widget Scaffolding Skill	3–4	☐
9	Ralph Loop Harness	3–4	☐
10	Plop.js Full Generator Suite	2–3	☐

#	Agent	Hrs	Status
11	Architecture MCP Query Server	8–12	☐
12	Playwright Canvas E2E	3–4	☐
13	Event Bus Integration Tests	2–3	☐
14	Storybook Widget Stories	2–3	☐
15	Supabase Migration Runner	1–2	☐
16	Vercel Preview Deployer	1–2	☐
17	commitlint Enforcer	0.5	☐
18	Visual Regression Testing	3–4	☐
19	Coverage Gap / Mutation Testing	2–3	☐
20	TypeDoc API Docs	2	☐
21	Mermaid Diagrammer	3–4	☐
22	Notion Doc Sync	3–4	☐
23	ADR Generator	2–3	☐
24	Release Automation	1–2	☐
25	ts-morph Codebase Analyzer	4–6	☐